An Autonomous Mobile-Manipulation Stack for Greenhouse Strawberry Harvesting: from a Webots Prototype to a ROS 2 / Gazebo Digital Twin

Franck Armand Josias Einstein
Projet de Fin d'Année (PFA)
Smart Agriculture — Digital Twin
Email: josiasarmand40@gmail.com

***Abstract*—This report documents the complete design, implementation and experimental evaluation of an autonomous mobile-manipulation stack for strawberry harvesting in a greenhouse, built around a KUKA YouBot omnidirectional platform. The work is presented chronologically in two phases. Phase A develops a self-contained prototype in Webots R2025a: mecanum kinematics, wheel odometry, log-odds occupancy mapping from a 2D lidar, A* path planning, pure-pursuit path following, colour-based crate detection with ground-plane back-projection, a five-degree-of-freedom arm sequence, and a behaviour-tree mission supervisor, totalling 3 844 lines across fifteen controllers. Phase B ports the validated algorithm layer to ROS 2 Jazzy and rebuilds the scene as a Gazebo Harmonic digital twin of a 9.90×4.90 m greenhouse containing three raised gutters and 127 individually placed berries. Phase B adds a single-owner protective stop, an offline UMBmark odometry calibration, a homemade correlative scan-matching SLAM module, a camera-to-map fruit localiser, a three-lap coverage survey preceding harvesting, and a measurement layer that scores pose, map and throughput against simulator ground truth on every run. Measured results are reported in full, including negative ones: the reconstructed map is accurate to -4 cm / $+10$ cm in interior dimensions with 100% floor coverage and 0.0% phantom clutter, calibrated odometry holds 0.20 m of position error where raw odometry diverges past 10 m, while the homemade SLAM front-end is shown to be worse than the odometry prior it was meant to correct, and the colour-threshold fruit detector reaches only 77% recall at 0.22 m localisation error with roughly two of every three map estimates spurious. Each algorithm and design decision is analysed with a SWOT table, and every control-flow structure is given as a colour-coded flowchart partitioned into initialisation, main loop and termination. The report also specifies the exact toolchain versions, installation procedure and launch commands required to reproduce every experiment.**

***Index Terms*—Autonomous mobile manipulation, precision agriculture, digital twin, ROS 2, Gazebo, Webots, KUKA YouBot, mecanum drive, occupancy grid mapping, SLAM, A*, pure pursuit, odometry calibration, functional safety, agricultural robotics.**

I. Introduction

A. Context and motivation

Strawberry production is among the most labour-intensive branches of horticulture. Harvesting alone accounts for a large share of the operating cost of a table-top greenhouse, it must be repeated every two to three days through the season, and it is increasingly constrained by the availability of seasonal labour. The task is also unusually hard to automate: the fruit is soft, it grows in clusters where ripe and unripe berries touch, it is partly hidden by leaves, and the picking window is short. These properties have made strawberry harvesting a recurring benchmark for agricultural robotics rather than a solved problem [**?**], [**?**].

A greenhouse, on the other hand, is a far friendlier environment for a mobile robot than an open field. It is flat, indoor, geometrically regular, free of weather, and its layout — parallel raised gutters separated by service aisles — is known in advance and does not change between seasons. That combination is what makes a *digital twin* attractive: because the physical structure is stable and measurable, a simulated greenhouse can be made dimensionally faithful, and a control stack developed against it has a realistic chance of transferring to the real installation.

B. Objective of this work

The objective of this Projet de Fin d'Année is to build, from nothing, a complete autonomous stack that lets a KUKA YouBot omnidirectional mobile manipulator:

1) build a metric map of an unknown greenhouse from a 2D lidar;
2) localise itself in that map while it moves;
3) plan and follow collision-free paths along the service aisles;
4) detect ripe strawberries with an on-board colour camera and place them on the map;
5) drive its five-DOF arm to pick them; and
6) repeat the cycle autonomously, delivering the fruit to a depot.

A secondary objective, imposed by the intended reuse of the platform in healthcare logistics and indoor transport, is that the navigation, mapping and safety layers must be *domain-independent*: nothing in them may assume strawberries, gutters, or this particular greenhouse.

C. Two phases, and why

The work is presented chronologically because it was carried out chronologically, and because the second phase is only intelligible as a consequence of the first.

Phase A (Webots). A complete prototype was first written for Webots R2025a, in plain Python, with no middleware. This was deliberate: Webots supplies the robot model, the physics and the sensors out of the box, so the entire effort could go into the algorithms themselves — mecanum kinematics, odometry, occupancy mapping, A*, pure pursuit, colour vision, arm sequencing, and a behaviour-tree mission supervisor. Fifteen controllers were written, ten of them single-purpose test controllers used to validate one subsystem at a time. Phase A produced 3 844 lines of Python and, more importantly, an algorithm layer that had been exercised against a working simulator.

Phase B (ROS 2 / Gazebo). The validated algorithms were then lifted, essentially unmodified, into a framework-independent library (`youbot_control/lib/`) and wrapped in ROS 2 Jazzy nodes, with the scene rebuilt as a Gazebo Harmonic digital twin measured against the real greenhouse. Phase B is therefore not a rewrite but a *re-architecture*: the same A*, the same pure pursuit, the same log-odds grid, now distributed across processes, with the problems that distribution creates — and with a measurement layer added so that those problems become visible instead of being guessed at.

D. What this report claims, and what it does not

This report is written to be checked. Every quantitative statement in it is traceable to a specific line of a specific simulation log, and the procedure to regenerate those logs is given in Appendix A. Where a subsystem does not work, that is stated with the same emphasis as where one does. In particular:

- The mapping subsystem works well. The reconstructed interior is accurate to -4 **cm /** $+10$ **cm** against a 9.90×4.90 m reference, with **100%** of the free floor observed and **0.0%** of it wrongly marked as obstacle (Sec. VII).
- Odometry calibration works. A one-off UMBmark-style calibration [**?**] holds position error at **0.20 m** over a full mission, where the uncalibrated odometry diverges past **10 m**.
- *The homemade SLAM module does not work.* Incremental correlative scan matching without loop closure was implemented, instrumented, and measured to be *worse* than the odometry prior it corrects. Sec. V-D explains why this is a property of the algorithm class and not a coding defect, and the production configuration disables it.
- *The fruit detector is not accurate enough.* Colour thresholding reaches **77%** recall but only about one correct estimate in three, at **0.22 m** localisation error — roughly a quarter of the plant spacing, which means it identifies the right plant but not the right berry.

Reporting the negative results in full is not modesty. The instrumented comparison between a homemade SLAM front-end and a calibrated dead-reckoning baseline is one of the more useful outcomes of the project, and it only exists because the measurement layer was built before the conclusions were drawn.

E. Contributions

1) A complete, documented, reproducible two-phase implementation of an autonomous greenhouse harvester, from world modelling to mission supervision, with every algorithm re-implemented rather than imported from a navigation stack.
2) A framework-independent algorithm library validated in one simulator (Webots) and reused unchanged in another (Gazebo/ROS 2), demonstrating a practical portability pattern.
3) A three-axis measurement layer — pose accuracy, map accuracy, and time/throughput — that scores the running system against simulator ground truth continuously, and which caught two regressions that no test and no visual inspection had caught.
4) A pre-flight regression suite of 191 pure-Python checks, each one encoding a failure the project actually suffered, that runs in about one second before every simulation.
5) An explicit SWOT analysis of every algorithm and architectural decision (Sec. VI), including the ones that were abandoned.

F. How to read the flowcharts

Every control-flow structure in this report is given as a flowchart using one fixed visual language, defined once in Fig. 1 and never redefined afterwards. Two conventions matter:

Shapes follow classic flowchart practice, with two additions specific to a robotics stack: a distinct shape for a *message read or written on a ROS topic*, and a distinct shape for a *call into the framework-independent algorithm library*. Being able to see, at a glance, where a diagram touches the middleware and where it touches the algorithms is the whole point of the split described in Sec. V-B.

Phase bands partition every diagram into three coloured regions: **INITIALISATION** (executed once, before any data arrives), **MAIN LOOP** (the periodic callback, executed at a fixed rate for the whole run), and **TERMINATION / DESTRUCTION** (shutdown, resource release, final reporting). The banding is not decoration: a recurring source of defects in this project was code placed in the wrong band — state armed once that should have been re-armed per cycle, and cleanup that never ran because it was inside the loop it was meant to end.

G. Report structure

Sec. II reviews the literature each algorithm is drawn from. Sec. III specifies the toolchain, the exact software versions, and the installation procedure. Sections IV–IV-M present Phase A (Webots) in full, subsystem by subsystem. Sections V–V-H

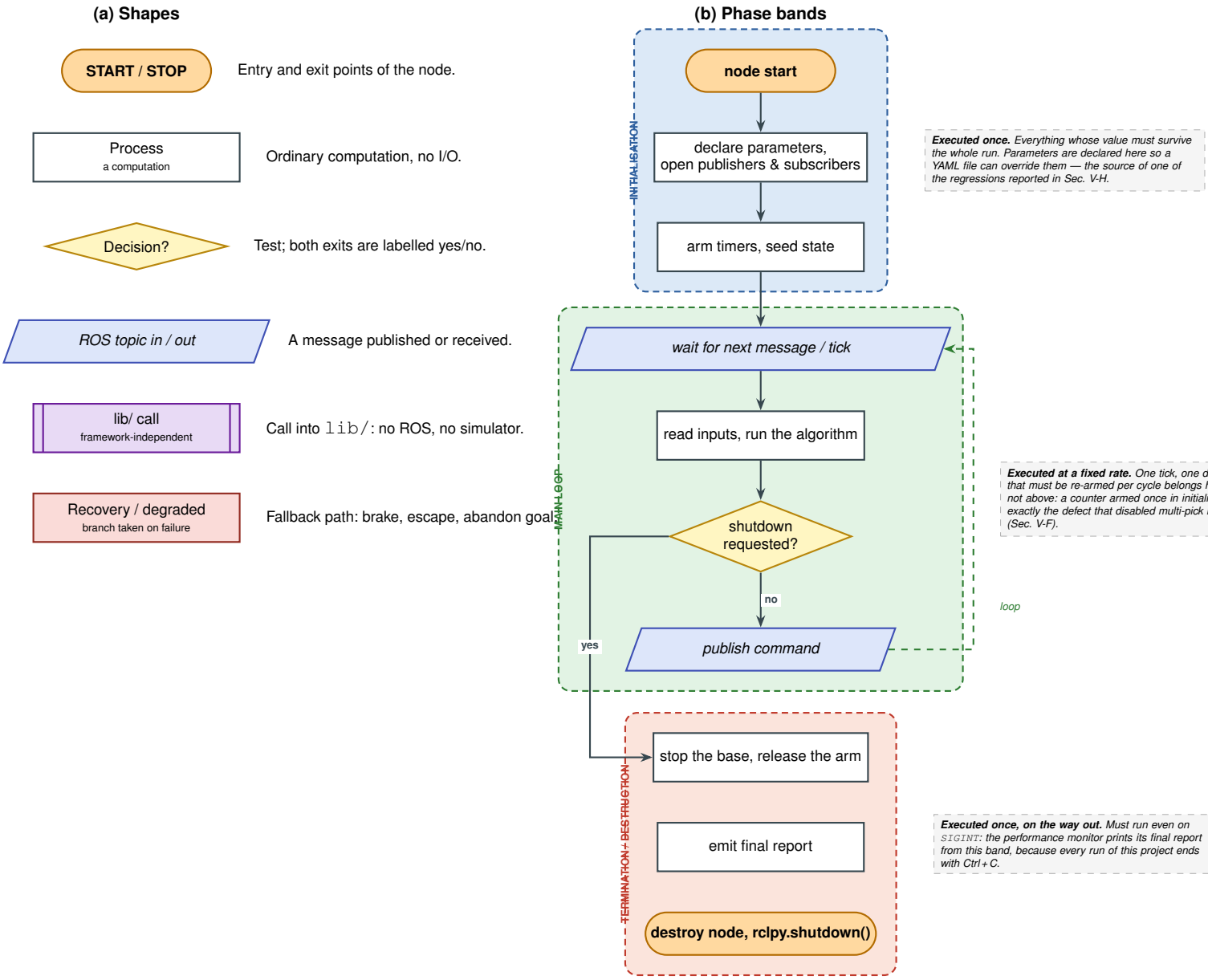


Figure 1: The flowchart language used throughout this report, defined once. (a) The six node shapes: two are specific to a robotics stack — a *ROS topic* shape, so the boundary with the middleware is visible, and a *library call* shape, so the boundary with the framework-independent algorithm layer is visible. (b) The three phase bands into which every later flowchart is partitioned. The banding is functional rather than decorative: two of the defects analysed in this report are cases of code placed in the wrong band.

present Phase B (ROS 2 / Gazebo) in the same order, so the two phases can be read side by side. Sec. VI gives the SWOT analysis of every method. Sec. VII reports the measured results, Sec. VIII discusses the limitations and what they imply for a physical deployment, and Sec. IX concludes. Appendix A gives the commands to reproduce every experiment; Appendix B tabulates every tuned parameter with its justification; Appendix C gives the full topic contract; Appendix D collects the failure modes encountered and their diagnosis.

II. Related Work

[Section in preparation.]

III. Development Environment and Toolchain

This section specifies the software environment precisely enough that every result in this report can be regenerated on a clean machine. It is placed before the technical chapters deliberately: several of the defects analysed later are consequences of version-specific behaviour (a deprecated Gazebo service, a build mode that resolves entry points through a symbolic

Table I: Development and experiment platform

Item	Value
Machine	HP EliteBook laptop
Operating system	Ubuntu 24.04 LTS (Noble Numbat)
Display server	Wayland, with Qt forced to XCB
Python	3.12 (system interpreter)
Project storage	external volume, mounted under `/mnt`

Table II: Software components and versions

Component	Version	Role
Webots	R2025a	Phase A simulator
ROS 2	Jazzy Jalisco	Phase B middleware
Gazebo Sim	Harmonic 8.11.0	Phase B simulator
`ros_gz_sim`	Jazzy	spawn, world launch
`ros_gz_bridge`	Jazzy	topic bridging
`webots_ros2_driver`	Jazzy	Phase A→ROS
RViz 2	Jazzy	visualisation
NumPy	system package	grids, scan matching
colcon	system package	build tool
Physics engine	DART (default)	1 ms step

link), and those are not intelligible without knowing exactly what was running.

A. Hardware and operating system

All development and all reported experiments were carried out on a single workstation, Table I. No GPU-accelerated inference is used anywhere in the stack; the only GPU consumer is the simulator's rendering pipeline, which also serves the simulated lidar (Sec. V-A explains why that matters).

The display server is worth recording. Gazebo's GUI detects Wayland and falls back to the XCB Qt platform plugin, printing `Detected Wayland. Setting Qt to use the xcb plugin` at every start. This is benign in itself, but it is one reason the GUI camera-tracking behaviour differs from the documented default, which motivated the camera watchdog described in Sec. V-A.

B. Software versions

Table II lists every component whose version affects the results. Two entries are load-bearing:

- **Gazebo Harmonic (gz-sim 8.11.0)** deprecated the `/gui/follow` service in favour of the `/gui/track` topic. Code written against the older API fails silently — the camera simply never locks onto the robot — which cost one full debugging cycle (Appendix D).
- **`colcon build --symlink-install`** resolves console-script entry points through an egg-link back into `src/`. When that metadata goes missing, every node dies at import with `PackageNotFoundError` — fifteen identical tracebacks that name the symptom and not the cause. The launch wrapper tests for this explicitly before starting anything.

C. Installation

Listing 1 gives the complete installation from a clean Ubuntu 24.04. The ROS 2 and Gazebo repositories are added first; the `ros-jazzy-ros-gz` metapackage pulls in both the bridge and the simulator integration.

```
# --- 1. ROS 2 Jazzy ------------------------------------
sudo apt update && sudo apt install -y \
     software-properties-common curl
sudo add-apt-repository universe
sudo curl -sSL -o /usr/share/keyrings/ros-archive-keyring.gpg \
  https://raw.githubusercontent.com/ros/rosdistro/master/ros.key
echo "deb [signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
http://packages.ros.org/ros2/ubuntu noble main" \
     | sudo tee /etc/apt/sources.list.d/ros2.list
sudo apt update
sudo apt install -y ros-jazzy-desktop

# --- 2. Gazebo Harmonic + the ROS 2 bridge -------------
sudo apt install -y ros-jazzy-ros-gz \
                    ros-jazzy-ros-gz-sim \
                    ros-jazzy-ros-gz-bridge

# --- 3. Webots R2025a (Phase A) ------------------------
sudo apt install -y ros-jazzy-webots-ros2
# plus the Webots R2025a .deb from cyberbotics.com

# --- 4. Build tooling and Python dependencies ----------
sudo apt install -y python3-colcon-common-extensions \
                    python3-numpy python3-opencv

# --- 5. The report toolchain (optional) ----------------
sudo apt install -y texlive-latex-extra texlive-publishers \
                    texlive-science
```

Listing 1: Full installation on a clean Ubuntu 24.04.

Table III: ROS 2 workspace packages and their dependencies

Package	Depends on
`youbot_control`	`rclpy`, `geometry_msgs`, `sensor_msgs`, `nav_msgs`, `tf2_ros`, `visualization_msgs`, `numpy`
`youbot_slam`	`rclpy`, `nav_msgs`, `sensor_msgs`, `geometry_msgs`, `tf2_ros`, `youbot_control`
`youbot_gazebo`	`youbot_control`, `youbot_bringup`, `ros_gz_sim`, `ros_gz_bridge`, `robot_state_publisher`, `rviz2`
`youbot_bringup`	`youbot_control`, `launch`, `launch_ros`, `rviz2`, `robot_state_publisher`
`youbot_webots`	`webots_ros2_driver`, `youbot_control`, `rclpy`

D. Workspace layout and build

The ROS 2 workspace contains five packages, Table III. The split is not cosmetic: `youbot_control` holds the algorithms and the control nodes and depends on *no* simulator, while `youbot_gazebo` and `youbot_webots` are thin, simulator-specific shells. This is what allowed Phase A to be reused in Phase B rather than rewritten (Sec. V-B).

Note the direction of the arrows: no package containing an algorithm depends on a simulator, and no simulator package is depended upon by another. `youbot_slam` depends on `youbot_control` only to reuse the occupancy-grid and scan-matching libraries.

```
git clone https://github.com/\
franckarmandjosiasEinstein-bit/ROS2_stage.git
cd ROS2_stage
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
```

Listing 2: Cloning and building.

`--symlink-install` is used so that editing a Python node takes effect without a rebuild, which matters when one simulation run costs half an hour of wall time. Its price is the entry-point fragility described above; if that appears, a plain `colcon build` writes real distribution metadata and removes the dependency on `src/` entirely.

E. Running a simulation

Every experiment is launched through one wrapper script, `src/youbot_gazebo/scripts/run.sh`, rather than through `ros2 launch` directly. The wrapper exists because of three concrete failures, each of which cost at least one run:

1) **Gazebo does not die on Ctrl + C.** A surviving `gz sim` keeps publishing an old `/clock`; the next run then has two clocks, RViz thrashes, and the GUI can attach to a ghost world with no robot in it. The wrapper installs a cleanup trap and afterwards *verifies* that nothing survived, printing the command line of any process it could not match rather than leaving it to be inferred from the next run misbehaving.
2) **Gazebo rewrites its GUI configuration on every exit** with wherever the camera was left, and reads it back in preference to the world's `<gui>` pose. A run started after the camera was parked on empty space opens blind. The wrapper deletes `~/.gz/sim/8/gui.config` before starting.
3) **Silent build breakage**, via the entry-point check described above.

The wrapper then runs the pre-flight regression suite (Sec. V-H) and refuses to start if any check fails: about one second of pure Python weighed against thirty minutes of simulation.

```
# Level 2 -- ground-truth pose, no SLAM (baseline)
./src/youbot_gazebo/scripts/run.sh

# Level 3 -- homemade SLAM in the loop
./src/youbot_gazebo/scripts/run.sh slam

# Level 3 -- calibrated dead reckoning, SLAM
    ↪ disabled
#            (the production configuration, Sec. V
    ↪ -E)
./src/youbot_gazebo/scripts/run.sh slam \
      slam:=false pose_topic:=odom_calibrated

# Headless, for batch runs
./src/youbot_gazebo/scripts/run.sh slam gui:=false
```

Listing 3: The supported launch configurations.

A correct start prints three markers, in this order. Their absence is the fastest way to detect that a stale build is being run — a mistake made twice during this project, both times producing a log that looked plausible and was measuring the wrong binary:

```
[run.sh] pre-flight: all 191 checks passed.
[mapping_node] mapping_node up: /scan + /odom -> /
    ↪ map
                (inflated, for planning) + /map_raw
    ↪  (evidence)
[perf_monitor] perf_monitor up: timing, throughput
    ↪  and
                where the time goes.
```

F. Running Phase A (Webots)

Phase A is self-contained and does not require ROS. Webots is opened on one of the two worlds and the `controller` field of the YouBot node selects which of the fifteen controllers runs (Sec. IV). The `youbot_webots` package exists only to expose the same robot to ROS 2 through `webots_ros2_driver`; it was used to confirm that the algorithm layer behaved identically under both simulators before the Gazebo twin was built.

```
# Standalone Webots (no ROS): open the world, then
    ↪  choose
# the controller in the scene tree.
webots worlds/smart_agriculture.wbt
webots worlds/greenhouse.wbt

# The same robot exposed to ROS 2 (bridge
    ↪ validation)
ros2 launch youbot_webots webots.launch.py
```

Listing 4: Launching Phase A.

G. Version control and reproducibility

The two phases live in two repositories: the Webots prototype in `Projet`, and the ROS 2 stack in `ROS2_stage`. Every result quoted in Sec. VII names the commit it was produced from, because two of the analyses in this report compare runs that differ by a single parameter file. Appendix A lists those commits alongside the exact command and the expected output.

IV. Phase A — Webots Prototype: Overview

A. Why a simulator-native prototype first

The first decision of the project was not which algorithm to use but *where to make the first mistake*. Two routes were available: start directly in ROS 2, where the target architecture lives, or start in Webots [**?**], where the robot model, the physics and the sensors are supplied and a controller is a single Python file with direct access to every device.

Webots was chosen, for three reasons that all proved correct in hindsight:

1) **The KUKA YouBot is a first-class Webots model.** Its mecanum base, its five-DOF arm and its two-finger gripper are modelled with the manufacturer's dimensions. No URDF had to be authored, no inertias guessed, no controller plugin written before the first wheel turned.
2) **One process, one debugger, no middleware.** A defect in a ROS 2 stack can be in the algorithm, in a QoS setting, in a remapping, in a transform, or in a launch file. In a Webots controller it can only be in the algorithm. Every hour spent debugging in Phase A was spent on robotics rather than on plumbing.
3) **Middleware is a re-architecture, not a rewrite.** If the algorithms are written without importing the simulator, moving them later is mechanical. Sec. V-B shows that this held: A*, pure pursuit, the log-odds grid, the mecanum kinematics and the scan matcher were carried into Phase B essentially unmodified.

The cost of the choice is that Phase A proves nothing about distributed execution, message timing, or transform trees — exactly the class of problem that dominated Phase B (Sec. V).

B. The Phase A mission

It is important to state plainly that Phase A and Phase B do not solve the same task. Phase A implements a *crate-collection* mission in a procedurally generated agricultural scene: the robot explores an arena containing six planting basins (`bassin_1`...`bassin_6`), discovers red crates with its camera, plans a route to each, picks it up with the arm, stows it in one of two body-frame slots on its rear plate, and delivers a full load to a depot before returning to explore again.

Phase B replaces the payload with strawberries growing on raised gutters. The mission logic changes; the navigation, mapping, planning and safety layers do not. That is precisely the domain-independence requirement stated in Sec. IV, and Phase A is where it was earned: nothing in `navigation.py`, `planning.py` or `mapping.py` mentions a crate.

C. Incremental validation: fifteen controllers

The distinguishing feature of Phase A is methodological rather than algorithmic. Instead of writing one controller and debugging it as a whole, *one single-purpose test controller was written per subsystem*, in dependency order, and each had to work in isolation before the next was started. Table IV lists all fifteen with their size.

Ten of the fifteen exist only to answer one question. This is deliberate. A test controller that fails tells you which subsystem is broken; an integrated controller that fails tells you only that something is. The discipline is also what made the port to Phase B tractable, because each module arrived at the port already having a known-good reference behaviour.

D. Module inventory

The mission controller itself is thin: it wires together eight modules, Table V, none of which imports Webots except through a small device-handle interface. The line counts are worth noting for what they say about where the difficulty lies: the behaviour tree and the localisation estimator are together larger than the planner, the mapper and the navigator combined.

Table IV: Phase A controllers, in the order they were written

Controller	LoC	Question it answers
Bring-up		
`device_scan`	61	What devices does this robot expose, and under what names?
`wheel_test`	99	Does each wheel turn, in the direction and at the rate commanded?
`base_test`	131	Do the mecanum equations produce forward, strafe and yaw correctly?
`odom_test`	115	Does wheel-encoder odometry track the ground-truth pose?
Perception		
`lidar_test`	149	Does the lidar produce a usable occupancy grid of a known scene?
`vision_test`	110	Are red crates segmented, and does the ground projection land on them?
`localize_test`	160	Does scan matching correct a deliberately drifted odometry?
Motion		
`planner_test`	115	Does A* return a collision-free path on the inflated grid?
`nav_test`	108	Does pure pursuit follow that path without cutting corners?
Manipulation		
`arm_test`	110	Does each arm joint reach its commanded angle?
`grasp_test`	170	Does the full pick sequence close on a crate and lift it?
Integration		
`world_manager`	185	Supervisor: generate the scene, score the mission, provide ground truth.
`my_First_ctrl`	603	The behaviour-tree mission that uses every module above.

Table V: Phase A algorithm modules

Module	LoC	Contents
`behavior_trees`	514	Blackboard, mission steps, and the py_trees behaviours
`mapping`	260	Log-odds occupancy grid, Bresenham ray casting, inflation
`localization`	252	Wheel odometry, likelihood-field scan matcher, fused estimator
`navigation`	236	Mecanum kinematics and pure pursuit
`vision`	169	Red-blob segmentation and ground back-projection
`planning`	139	A*, octile heuristic, path smoothing
`exploration`	96	Frontier extraction and waypoint ordering
`arm`	62	Joint poses, gripper open/close
Total (modules)	**1 728**	
Total (Phase A)	**3 844**	including all fifteen controllers

E. Reading order for this chapter

The subsections that follow take the subsystems in the order they were built, which is also the order of Table IV: the world and its supervisor (Sec. IV-F), locomotion and odometry (Sec. IV-G), perception (Sec. IV-H), mapping and localisation (Sec. IV-I), planning and path following (Sec. IV-J), manip-

ulation (Sec. IV-K), and finally the behaviour-tree mission that binds them (Sec. IV-L). Sec. IV-M reports what Phase A achieved and, more usefully, which of its design decisions had to be undone in Phase B.

F. World Modelling and the Procedural Greenhouse

1) Two worlds, two purposes: Phase A uses two Webots worlds. `smart_agriculture.wbt` (258 lines) is the working scene: a 10 m square arena with a rectangular floor, four rows of planting basins, a robot start corner and a depot corner. `greenhouse.wbt` (100 lines) is a reduced scene used by the single-subsystem test controllers, where a smaller and fully deterministic layout makes a failure easier to attribute.

2) Why the scene is generated, not authored: The decisive design choice in Phase A was to make the working scene *procedural*. A supervisor controller, `world_manager.py` (185 lines), rebuilds the arena at the start of every run from a fresh random seed, which it prints so that any run can be replayed:

```
seed = random.randrange(1_000_000_000)
random.seed(seed)
print(f"[world_manager] Greenhouse seed = {seed}")
```

The alternative — one hand-authored scene — is faster to build and much more dangerous. A controller developed against a fixed layout learns the layout. Waypoints get tuned until they happen to thread between two particular basins; a planner inflation radius gets reduced until one particular gap becomes passable. None of that survives contact with a different arrangement, and none of it is visible while the scene never changes. Randomising the scene makes overfitting fail loudly and early, on the developer's own machine.

3) What is randomised: Table VI lists the generation parameters. Three of them carry most of the variability:

- `POT_SKIP_PROB` $= 0.25$ deletes a quarter of the planting basins at random. The rows therefore have gaps, and those gaps change every run — so the planner must actually search rather than follow a memorised corridor.
- `N_CRATES` $\in [2, 5]$ makes even the number of mission objectives unknown at start-up, which is what forced the mission layer to be a discovery loop instead of a fixed sequence (Sec. IV-L).
- `N_WORKERS` $= 2$ adds moving obstacles that wander between random aisle points at 0.30 m/s. They are the only dynamic elements in Phase A, and they are what justifies re-running the mapper during navigation instead of mapping once and planning on a frozen grid.

The `KEEP_CLEAR` mechanism deserves a note because it is the kind of detail that is invisible until it bites. Nothing is spawned within 1.0 m of the robot's start corner $(-4.5, 4.5)$ or the depot $(4.5, -4.5)$. Without it, roughly one run in ten began with a crate spawned underneath the robot or a basin blocking the depot, and the resulting mission failure looked like a planner defect rather than a scene-generation artefact. A generator that can produce an impossible scene will eventually produce one, on the run being demonstrated.

Table VI: Procedural scene parameters (`world_manager.py`)

Parameter	Meaning	Value
`ARENA_HALF`	arena half-extent	5.0 m
`ROW_YS`	basin row centre lines	$\pm 1.0, \pm 3.0$
`POT_XS`	basin positions per row	7 at 1.2 m
`POT_SKIP_PROB`	probability a basin is omitted	0.25
`AISLE_YS`	free corridors	$-2.0, 0.0, 2.0$
`N_CRATES`	mission objectives	2–5
`N_WORKERS`	moving obstacles	2
`WORKER_SPEED`	obstacle speed	0.30 m/s
`KEEP_CLEAR`	protected zones	start, depot
`CLEAR_RADIUS`	protected radius	1.0 m

4) The supervisor's second role: ground truth: A Webots supervisor can read the true pose of any node in the scene graph. `world_manager.py` uses this for two things that a robot may not do:

1) **Scoring.** It knows how many crates it created and where they are, so it can report how many the robot found, how many it delivered, and how many it missed — an objective mission score that does not depend on the robot's own beliefs.
2) **Reference data for perception.** It can answer "what is really there?" for a detection, which is how the vision module's false-positive rate was measured rather than estimated.

This separation — the robot's belief on one side, the simulator's truth on the other, compared by a third party that participates in neither — is the single most valuable idea Phase A contributed to Phase B. It reappears there, considerably expanded, as the instrumentation layer of Sec. V-G: three nodes that continuously score pose, map and throughput against Gazebo ground truth. The measurement discipline of this project starts here.

5) Coordinate conventions: The world frame is the XY plane with its origin at the arena centre, $+x$ east and $+y$ north, following REP-103 [**?**]. Body-frame twists use x forward, y left, ψ counter-clockwise. The occupancy grid uses image conventions — row indexes y, column indexes x, origin at the top-left — so a vertical flip is applied whenever a grid is published or drawn. Getting that flip wrong produces a map that is a perfect mirror of reality and looks entirely plausible until a path is planned on it; it is checked explicitly in Phase B's regression suite (Sec. V-H).

G. Locomotion: Mecanum Kinematics and Odometry

1) The platform: The KUKA YouBot base carries four mecanum wheels [**?**], each fitted with a ring of passive rollers set at 45° to the wheel plane. A conventional wheel can exert force only along its rolling direction; a mecanum wheel, because its contact patch is a roller free to slide along its own axis, exerts force along a single diagonal. Four such wheels with alternating roller handedness span the full planar velocity space, so the base is *holonomic*: it can translate in any direction and rotate, independently and simultaneously.

This matters more for this application than it may appear. A greenhouse service aisle in the Phase B twin is 0.80 m wide

Table VII: Mecanum platform constants (Phase A)

Symbol	Meaning	Value
r	wheel radius	0.050 m
l_x	longitudinal centre-to-wheel	0.228 m
l_y	lateral centre-to-wheel	0.158 m
L	$l_x + l_y$	0.386 m
$\omega_{\max}$	commanded wheel ceiling	14.0 rad/s

while the base is 0.58×0.38 m; a differential-drive robot would have to perform a three-point turn to change rows, whereas the YouBot can strafe sideways out of a row without changing heading. Sec. V-C shows that this property became the basis of the escape manoeuvre after a rotating recovery was measured to be actively harmful.

2) Inverse kinematics: Let the body frame follow REP-103 [**?**]: x forward, y left, ψ yaw counter-clockwise. With wheel radius r, longitudinal half-spacing l_x and lateral half-spacing l_y, and writing $L = l_x + l_y$, the four wheel angular velocities $\omega_{1..4}$ that realise a commanded body twist (v_x, v_y, ω_z) are

$$\omega_1 = \tfrac{1}{r}\left(v_x + v_y + L\,\omega_z\right) \quad \text{(front left)} \tag{1}$$
$$\omega_2 = \tfrac{1}{r}\left(v_x - v_y - L\,\omega_z\right) \quad \text{(front right)} \tag{2}$$
$$\omega_3 = \tfrac{1}{r}\left(v_x - v_y + L\,\omega_z\right) \quad \text{(rear left)} \tag{3}$$
$$\omega_4 = \tfrac{1}{r}\left(v_x + v_y - L\,\omega_z\right) \quad \text{(rear right)} \tag{4}$$

with the measured platform constants of Table VII. The sign pattern is the one published in the Cyberbotics YouBot reference implementation, keyed by the *native* device index `wheel1...wheel4`. Keying by native index rather than by physical corner is a small decision that removed a whole class of bug: no remapping table exists, so no remapping table can be wrong.

3) Saturation that preserves direction: Equations (1)–(4) can demand a wheel speed the motor cannot deliver, and the YouBot ceiling is about 14.8 rad/s. The naive remedy — clipping each wheel independently — is wrong, and visibly so: clipping one wheel of a diagonal pair while leaving the other changes the *direction* of the resulting twist, so a robot asked to strafe left while turning drifts forward instead. The implementation therefore computes the peak demand and, if it exceeds the ceiling, scales *all four* wheels by the same factor:

$$\omega_i \leftarrow \omega_i \cdot \min\left(1, \frac{\omega_{\max}}{\max_j |\omega_j|}\right), \qquad i = 1 \ldots 4. \tag{5}$$

The robot then moves more slowly than asked, but in exactly the commanded direction. This is the mecanum analogue of the general principle that a saturation should preserve the shape of a command; the same principle reappears in Phase B, where the protective stop scales the translation by a factor rather than truncating one axis (Sec. V-C).

4) Forward kinematics and odometry: Odometry inverts the same relation. Given the four measured wheel increments $\Delta\theta_i$ over one control period, the body displacement is

$$\Delta x = \tfrac{r}{4}\left(\Delta\theta_1 + \Delta\theta_2 + \Delta\theta_3 + \Delta\theta_4\right) \tag{6}$$
$$\Delta y = \tfrac{r}{4}\left(\Delta\theta_1 - \Delta\theta_2 - \Delta\theta_3 + \Delta\theta_4\right) \tag{7}$$
$$\Delta\psi = \tfrac{r}{4L}\left(\Delta\theta_1 - \Delta\theta_2 + \Delta\theta_3 - \Delta\theta_4\right) \tag{8}$$

integrated into the world frame with the mid-point heading $\psi + \Delta\psi/2$ to avoid the first-order bias of using the heading at the start of the interval.

Two properties of this estimator dominate everything built on top of it, and both were measured in `odom_test`:

- **It is exact in the absence of slip.** Over short, straight motions the estimate tracks the ground-truth GPS to within a few millimetres.
- **Its error is unbounded and monotone.** Mecanum rollers slip by construction, particularly during rotation, and every slip is integrated and never forgotten. This is not a defect to be tuned away; it is the reason localisation exists, and it is why Phase B devotes an entire subsystem to bounding it (Sec. V-D).

5) Control flow: Fig. 2 gives the base controller as it runs in `base_test` and, unchanged in substance, inside the mission controller. The banding is instructive: everything that can be computed once — device handles, the constant L, putting the wheel motors into velocity mode by setting their position target to infinity — is in the initialisation band, and the main loop contains nothing but the kinematics and the saturation. The termination band exists because a Webots controller that exits without zeroing the wheel velocities leaves the robot coasting into the next run.

6) Validation: `wheel_test` commands each wheel in isolation and verifies sign and rate against its position sensor. `base_test` commands the three canonical twists — pure forward, pure strafe, pure yaw — and checks that the GPS displacement matches the commanded direction to within a tolerance. `odom_test` drives a closed rectangle and reports the loop-closure error, which is the quantity later used to calibrate the systematic component of the error in Phase B (Sec. V-D). Running the three in order takes under two minutes and was repeated after every change to the kinematics.

H. Perception: Lidar and Colour Vision

Phase A carries exactly two exteroceptive sensors, and they answer two different questions. The lidar answers *"where can I go?"*; the camera answers *"what is that, and where is it?"*. Neither can answer the other's question, and a substantial part of the mission logic (Sec. IV-L) exists to reconcile the two.

1) The lidar: what the sensor physically is: The sensor is declared in the robot's `bodySlot` as

```
Lidar {
  translation 0 0 0.2
  name "lidar"
  horizontalResolution 360
  fieldOfView 6.28
  numberOfLayers 1
  maxRange 12
}
```

Read literally, this is a single horizontal plane, 0.20 m above the base origin, sampled at 360 bearings over a full 6.28 rad revolution, with a 12 m ceiling. Four consequences follow directly from those five numbers, and every one of them shaped a later design decision:

1) **It is a slice, not a view.** The lidar sees only what intersects one horizontal plane at one height. An obstacle below that plane — a crate lying on the floor — is invisible to it, which is precisely why crate detection had to be done in the camera and not in the lidar.
2) **Its angular resolution is 1°.** Two returns 2 m away are 3.5 cm apart; at 5 m they are 8.7 cm apart. A gap narrower than the beam spacing at that range is simply not resolved, so the far field of a scan is systematically coarser than the near field. The mapper inherits this: a distant cell receives fewer and more widely spaced updates, and therefore converges more slowly — which is the correct behaviour, not a defect.
3) **It measures range, not free space.** A beam that returns 3 m asserts one occupied point *and* a 3 m segment of emptiness behind it. Turning that pair of assertions into a map is the entire subject of Sec. IV-I.
4) **Its origin is the base centre, not the bumper.** The sensor sits at the geometric centre of a 0.58×0.38 m body. A raw range of 0.28 m straight ahead therefore describes a point *inside* the robot's own nose. This single fact caused the most damaging defect of the whole project, in Phase B, and is dissected in Sec. V-C.

a) A common misconception, stated and corrected: A 360° lidar is often described as giving a robot "awareness of everything around it". It does not. It gives a *complete horizontal ring of distances at one height, with no identity attached to any of them.* The scan cannot distinguish a wall from a plant gutter from a moving worker; it cannot see above or below its own plane; it has no memory from one scan to the next; and it produces no notion of "a corridor" or "a row" — those are inventions of the mapper and the planner. Every behaviour that resembles spatial understanding in this system is software built on top of a ring of numbers.

2) The camera: intrinsics as declared:

```
DEF CAMERA_RGB Camera {
  translation 0.2 0 0.3
  rotation 0 1 0 0.5
  name "camera"
  width 160  height 120
  fieldOfView 1.2
}
```

The mount is fixed and known: 0.20 m ahead of the base centre, 0.30 m above the body slot (about 0.40 m above the floor), pitched 0.5 rad downward about the body y axis. The image is 160×120 with a horizontal field of view of 1.2 rad. From these the pinhole intrinsics follow with no calibration step at all:

$$f_x = f_y = \frac{W/2}{\tan(\theta_{\text{fov}}/2)} = \frac{80}{\tan 0.6} \approx 118.6\,\text{px}, \tag{9}$$

with the principal point at $((W-1)/2,\ (H-1)/2)$. Square pixels are assumed, which is exact for a Webots camera.

3) Detection stage 1 — brightness-invariant red segmentation: Webots returns the image as a BGRA byte buffer, rows top to bottom. The first version of the detector used an absolute margin: a pixel was red if $R - G > 60$ and $R - B > 60$. It worked on the test bench and failed in the arena, because the checkerboard floor is itself a brownish red. The margin that rejected the floor also rejected any crate in shadow, where all three channels scale down together.

The fix was to threshold on *channel ratio* rather than difference:

$$\mathcal{M}(u,v) = \big[R > 60\big] \wedge \big[R \geq G\big] \wedge \big[R \geq B\big] \wedge \big[G < 0.33\,R\big] \wedge \big[B < 0.33\,R\big] \tag{10}$$

The measured pixel statistics justify the constant. A crate pixel has $G/R \approx 0.12$; a floor pixel has $G/R \approx 0.40$. A threshold of 0.33 separates them with margin on both sides and — this is the point of the change — a ratio is invariant under illumination scaling, so the same threshold holds in shadow where an absolute margin does not. The implementation is four vectorised comparisons over the whole array:

```
limit = self.red_ratio * r
return ((r > self.value_min) & (r >= g) & (r >= b)
        & (g < limit) & (b < limit))
```

4) Detection stage 2 — connected components: The mask is grouped into blobs by 4-connectivity flood fill, implemented with an explicit stack rather than recursion, because a 160×120 mask can produce a component deep enough to exhaust the Python recursion limit. Components smaller than `min_blob_pixels` $= 4$ are discarded as noise. That threshold is deliberately low: a crate at 3 m occupies only a handful of pixels in a 160×120 image, and raising the threshold to a comfortable-looking 20 pixels made the robot blind beyond 1.5 m. Each surviving component contributes its centroid $(\bar{u}, \bar{v})$ in pixel coordinates.

5) Detection stage 3 — back-projection onto the ground plane: A single camera cannot recover depth. It can, however, recover position *given a plane constraint*, and every crate rests on the floor at a known height $z_g = 0.05$ m. The centroid is first turned into a ray in the camera frame (Webots convention: $+x$ forward, $+y$ left, $+z$ up),

$$\mathbf{d}_c = \left(1,\ -\frac{\bar{u} - c_x}{f_x},\ -\frac{\bar{v} - c_y}{f_x}\right), \tag{11}$$

then rotated into the world by the camera's world orientation, which is *composed* rather than read from the simulator:

$$\mathbf{R} = \mathbf{R}_z(\psi)\,\mathbf{R}_y(\beta), \qquad \mathbf{p} = \begin{pmatrix} x_r + d\cos\psi \\ y_r + d\sin\psi \\ h_c \end{pmatrix}, \tag{12}$$

with ψ the robot yaw taken from its own pose estimate, $\beta = 0.5$ rad the fixed mount pitch, $d = 0.20$ m the mount offset and $h_c = 0.40$ m the camera height. Intersecting the

world-frame ray $\mathbf{d}_w = \mathbf{R}\,\mathbf{d}_c$ with the ground plane gives the scale and the estimate:

$$t = \frac{z_g - h_c}{d_{w,z}}, \qquad (x, y) = \big(p_x + t\,d_{w,x},\ p_y + t\,d_{w,y}\big). \quad (13)$$

a) Why the camera pose is composed and not read: The supervisor API could return the camera node's true world transform in a single call. Using it would have been shorter, and it would have been cheating: the estimate would then be exact regardless of how badly the robot's own localisation was drifting, and the perception error measured in `vision_test` would have been meaningless. Composing the pose from the robot's *own* estimate and a fixed, documented extrinsic keeps the whole detection chain onboard, and makes localisation error propagate into object position error exactly as it would on hardware. This is the same discipline that later separates `truth_monitor` from the robot in Phase B (Sec. V-G).

6) Three rejection gates: Equation (13) is numerically unstable near the horizon: as $d_{w,z} \to 0$ the intersection runs to infinity, so a one-pixel error in $\bar{v}$ can move the estimate by tens of metres. Three gates bound the damage, and each was added in response to an observed failure rather than anticipated:

- `min_ray_down` $= 0.12$ — reject rays not pointing down steeply enough. This is the horizon guard.
- `max_range` $= 3.0\,\mathrm{m}$ — reject ground hits beyond the distance at which a crate is more than a few pixels wide.
- `arena_bound` $= 4.6\,\mathrm{m}$ — reject any estimate outside the arena. A detection that lands inside a wall is wrong by construction, whatever produced it.

7) Two defects found only by testing against truth: `vision_test` spins the robot in place for 30 s and prints, once a second, the detections beside the supervisor's true crate positions. Two defects surfaced that no amount of staring at the mask would have revealed.

One-off false positives. A single frame in which a moving worker's red-brown texture passed Eq. (10) was enough to register a phantom crate, after which the mission would drive across the arena to collect nothing. The fix was *temporal confirmation*: a detection must be seen in several consecutive frames within a small spatial window before it is admitted to the crate list. The cost is a short delay before a real crate is accepted; the benefit is that uncorrelated single-frame noise is rejected outright.

Detections sitting on lidar obstacles. The remaining false positives clustered on the workers, which are both reddish and mobile. The fix was cross-modal: at discovery time, a candidate whose projected ground position falls close to a lidar return is rejected, because a crate lies below the lidar plane and therefore *cannot* produce one. The two sensors disagreeing is itself the evidence. This is the earliest appearance in the project of a principle that runs through all of Phase B: a belief supported by one sensor and contradicted by another should be dropped, not averaged.

8) What was measured: Across the `vision_test` runs the detector found every crate that entered the field of view within 3 m, and the residual position error was dominated not by the image processing but by the yaw term in Eq. (12): a 5° heading error at 2 m range displaces the estimate by 17 cm. That sensitivity is the reason Phase B devotes an entire subsystem to bounding heading drift (Sec. V-D), and it is why the Phase B fruit localiser inherits both the strength of this design — it works, cheaply, with no training data and no labelled dataset — and its weakness: it is never better than the pose that feeds it.

I. Mapping and Localisation

1) The representation: a log-odds occupancy grid: The map is a 100×100 grid covering the 10 m arena at 0.10 m resolution, following Moravec and Elfes [**?**]. Each cell carries not a binary state but a *log-odds* value ℓ, the logarithm of the ratio between the probability that the cell is occupied and the probability that it is free:

$$\ell(c) = \log \frac{P(c \text{ occupied})}{P(c \text{ free})}. \quad (14)$$

The reason for the logarithm is arithmetic, not aesthetic. In probability space, fusing independent observations requires a product and a renormalisation; in log-odds space it is an addition [**?**]. Fusing a scan therefore becomes a sweep of `+=` over an array, which is both fast and numerically untroubled. Two constants define the sensor model actually used:

`L_OCC`	added where a beam terminates	$+0.85$
`L_FREE`	added to every cell the beam crosses	-0.40
`L_CLAMP`	saturation bound on $\lvert\ell\rvert$	5.0
`L_OCC_THRESHOLD`	ℓ above which a cell is "occupied"	0.5

Three deliberate asymmetries are encoded in those four numbers.

Occupied evidence is worth more than free evidence (0.85 against 0.40). A beam that stops has made a positive measurement of a surface; a beam that passes through has only failed to find one. Making them equal produces a map that erodes thin obstacles, because a gutter edge is crossed by many more passing beams than terminating ones.

The belief is clamped. Without `L_CLAMP`, a wall seen five hundred times reaches $\ell = 400$ and needs a thousand contradicting observations to be forgotten. The map would then be unable to react to a moving worker who parks somewhere and later leaves. The clamp at ± 5 keeps roughly ten contradicting scans sufficient to flip a cell, which is the right time constant for obstacles moving at 0.3 m/s in a 10 m arena.

The occupancy threshold is not zero but $+0.5$. A cell must accumulate net positive evidence, not merely non-negative evidence, before the planner is told to avoid it. This is the cheapest available defence against single-beam noise.

2) Integrating one scan: exact ray casting: For each of the 360 beams the mapper computes the world-frame bearing $\alpha = \psi + \big(\frac{\text{fov}}{2} - i\,\Delta\big)$, decides whether the beam terminated on a surface (`dist` finite and below `max_range`), and walks the integer line from the robot cell to the endpoint cell with Bresenham's algorithm [**?**]:

```
for (cc, cr) in self._bresenham(r0c, r0r, ec, er):
    if cc == ec and cr == er:
        break            # stop BEFORE the
    ↪ endpoint
    self.log_odds[cr, cc] += L_FREE
if hit:
    self.log_odds[er, ec] += L_OCC
```

The `break` is not decoration. Without it the endpoint cell receives `L_FREE` on the same pass that gives it `L_OCC`, and the wall's net evidence per scan falls from $+0.85$ to $+0.45$ — halving the convergence rate for no reason at all. It is a one-line detail that is invisible in the code review and obvious in the resulting map.

a) A rejected optimisation, and why it was rejected: The per-beam Python loop is the slowest part of Phase A. A numpy rewrite was implemented and benchmarked: sample each ray at half-cell intervals and scatter with `np.add.at`. It was measured to be *both slower and less accurate* — slower because `np.add.at` is not vectorised internally over duplicate indices, and less accurate because half-cell sampling can skip the very cell a beam grazes, eroding occupied cells along oblique walls. The exact sweep was kept, and the mapper was instead run once every `INTEGRATE_EVERY` $= 4$ control ticks, which costs nothing in map quality because the robot moves less than 4 cm in that interval. Recording a rejected optimisation is as much part of the engineering record as recording an accepted one.

b) A near-range gate: Beams shorter than `min_range` $= 0.25$ m are discarded entirely. At that distance the return is either the robot's own structure or noise, and integrating it stamps a permanent obstacle onto the cell the robot is standing in — after which the planner cannot find a path out of it.

3) From belief to plan: thresholding and inflation: The planner cannot consume log-odds. `update_grid_from_log_odds` produces the binary planning grid in two steps: threshold at `L_OCC_THRESHOLD`, then dilate every occupied cell by a disc of radius `inflation`.

Inflation is what turns a map of the *world* into a map of the *robot's configuration space* [**?**]. Once obstacles are grown by the robot's own radius, the planner may treat the robot as a dimensionless point, and any path it returns through free cells is collision-free for the real body. In Phase A the radius is 0.32 m, chosen to cover the YouBot half-diagonal of $\sqrt{0.29^2 + 0.19^2} = 0.347$ m with a small deliberate shortfall so that the 1.2 m aisles remain traversable.

That choice — a disc sized on the half-diagonal — is exactly the approximation that Phase B later had to abandon. A disc of 0.347 m is correct for a robot that may be rotated arbitrarily, but it is 16 cm too conservative sideways, and in the 0.80 m aisles of the Phase B greenhouse that conservatism is the difference between a passable lane and an impassable one. The rectangle model of Sec. V-C replaces it.

The dilation itself is done by shifted whole-array ORs rather than a per-cell neighbourhood scan:

```
for dr in range(-radius, radius + 1):
    for dc in range(-radius, radius + 1):
        if dr*dr + dc*dc > radius*radius:
            continue
        out[r_dst0:r_dst1, c_dst0:c_dst1] |= \
            occupied[r_src0:r_src1, c_src0:c_src1]
```

which is $O(\text{radius}^2)$ array operations instead of $O(\text{cells} \times \text{radius}^2)$ scalar ones — for a 100×100 grid and a 3-cell radius, 45 array ORs in place of 450 000 Python iterations.

4) Localisation, part 1: dead reckoning: `localization.py` first provides a GPS-free pose by integrating the mecanum forward kinematics of Sec. IV-G over the four wheel encoders. It is exact in the absence of slip and its error is unbounded and monotone in its presence. Every metre driven adds error that is never removed, which is the entire justification for what follows.

5) Localisation, part 2: likelihood-field scan matching: The corrector is a *correlative scan matcher* [**?**]: given a pose prior from odometry, search a small window of $(\Delta x, \Delta y, \Delta\psi)$ offsets for the one that best explains the current scan against a reference map. The score of a candidate pose is a likelihood field [**?**]: project the scan endpoints into the map, look up each endpoint's distance d to the nearest obstacle surface, and sum a Gaussian,

$$S(x, y, \psi) = \sum_{k \in \text{beams}} \exp\left(-\frac{d_k^2}{2\sigma^2}\right), \qquad \sigma = 0.15\,\text{m}. \quad (15)$$

Three implementation decisions carry the performance, and each of them was made after the naive version failed.

a) The distance field is seeded from surfaces, not from solids: The first version computed distance-to-nearest-*occupied-cell*. This saturates: a scan shifted bodily *inside* a large planting basin still lands every endpoint on an occupied cell, scores a perfect $d = 0$ everywhere, and the search has no gradient to climb. Real lidar only ever sees obstacle *boundaries*, so the breadth-first distance transform is seeded from occupied cells that touch free space:

```
occ = grid > 0
free_neighbor[:-1, :] |= ~occ[1:, :]   # and the
    ↪ three other shifts
return occ & free_neighbor
```

Endpoints that drift into an obstacle interior are then penalised, and the score surface acquires a genuine peak at the true pose.

b) Coarse-then-fine, with a cached beam layout: The search runs a coarse pass (± 0.30 m in steps of 0.10 m, ± 0.10 rad in steps of 0.05 rad) followed by a fine pass around the winner (± 0.06 m / 0.02 m, ± 0.04 rad / 0.02 rad). That is $7^3 = 343$ candidates plus 7^3 again, rather than the $31^3 = 29\,791$ a single fine sweep would need. Every beam is subsampled by `beam_stride` $= 6$ (60 of 360 beams), and the subsampled indices and their local bearings depend only on the scan geometry — constant across all 686 candidates — so they are computed once and cached. The score itself is fully vectorised, which measured about ten times faster than the equivalent per-beam loop.

c) A strictly-better trust gate: The corrected pose is accepted *only if it scores strictly higher than the prior*:

```
if score <= prior_score:
    return prior, prior_score
```

The reason is a failure that was observed rather than predicted. In a region of long straight walls the score surface has a broad plateau: a scan shifted along the wall explains the map exactly as well as the true pose. Without the gate the search settles on an arbitrary point of that plateau, the result is fed back into the odometry as a reset, and the next iteration starts from a slightly worse prior — a slow runaway driven entirely by ties. Comparing with $\leq$ rather than $<$ also covers the degenerate case of an uninformative scan, where every candidate scores identically and the search would otherwise adopt a window-corner offset on no evidence at all.

This single line is, in retrospect, the most important defensive decision in the localisation stack, and its Phase B descendant is the rule that a scan-matching estimate may never be trusted merely because it exists (Sec. V-D, where the homemade matcher is shown to be *worse* than the prior it was meant to improve).

6) Assembling the estimator: `LocalizationEstimator` bundles the two behind one `pose()` callable: odometry integrates every tick, and every `correct_every` $= 8$ ticks the scan matcher snaps the estimate back onto the reference map and resets the odometry to the corrected value. Between corrections the pose tracks the wheels; the drift is bounded by the correction period rather than by the distance driven. Because the interface is a single callable, the mission layer can be switched between ground-truth GPS and this estimator by changing one argument — which is how the two were compared quantitatively.

7) Frontier extraction: deciding where to look next: `exploration.py` closes the loop between the map and the mission. Rather than patrolling a fixed waypoint list, the robot drives to the boundary between known-free and unknown space [**?**]. Working directly on the log-odds array,

$$\text{free}(c) \iff \ell(c) < -0.5, \tag{16}$$

$$\text{unknown}(c) \iff |\ell(c)| < 0.1, \tag{17}$$

$$\text{frontier}(c) \iff \text{free}(c) \wedge \exists\, c' \in \mathcal{N}_4(c) : \text{unknown}(c'). \tag{18}$$

The three-way test is why the log-odds array is kept rather than discarded after thresholding: a binary grid cannot distinguish "observed and empty" from "never observed", and frontier exploration is exactly the algorithm that needs that distinction.

Frontier cells are clustered by 4-connectivity, clusters smaller than `min_cluster` $= 4$ cells are dropped as noise, and each surviving cluster contributes one goal. Two refinements matter:

- The goal is the frontier cell *nearest the robot*, never the cluster centroid. For a ring-shaped frontier — the normal shape early in a run — the centroid falls in the middle of already-explored space, and the robot is sent to a place it has already been.
- Cells closer than `min_dist` $= 0.6$ m are skipped, otherwise the robot twitches in place chasing a frontier it is already dissolving.

The resulting goals are ordered by a greedy nearest-first tour, and an empty list is the natural termination condition: no frontier means everything reachable has been seen. In Phase B this adaptive scheme is replaced by a deterministic three-lap boustrophedon survey (Sec. V-F) — not because frontier exploration failed, but because a fixed structured aisle layout makes a coverage pattern both simpler and time-bounded, and the report's time axis (Sec. VII) made bounded completion time a first-class requirement.

J. Planning and Path Following

Planning and following are two different problems and are solved by two different pieces of code. The planner answers *"what sequence of free cells connects here to there?"* once, on a static snapshot of the map. The follower answers *"what should the wheels do in the next 32 ms?"* continuously, on a pose that is always slightly wrong. Conflating them — a single controller that both searches and steers — is a recurring failure mode in student robotics projects, and this project deliberately avoided it from the first commit.

1) A* on the inflated grid: The planner is A* [**?**] over the 8-connected 100×100 inflated grid produced in Sec. IV-I. Straight moves cost 1, diagonal moves cost $\sqrt{2}$, and the heuristic is the *octile* distance, which is the exact cost of the cheapest unobstructed 8-connected path:

$$h(a,b) = \sqrt{2}\,\min(\Delta_x, \Delta_y) + \big|\Delta_x - \Delta_y\big|, \qquad \Delta_x = |a_x - b_x|. \tag{19}$$

The choice of heuristic is not cosmetic. Manhattan distance ($\Delta_x + \Delta_y$) *over*-estimates the cost of a diagonal move — it charges 2 where the true cost is 1.414 — and an inadmissible heuristic destroys A*'s optimality guarantee, which in practice shows up as paths that take an unnecessary dog-leg around a basin. Euclidean distance is admissible but loose on a grid that cannot move at arbitrary angles, and a loose heuristic expands more nodes than necessary. Octile is both admissible and tight for this connectivity, which is exactly the property one wants: optimal paths, minimal expansions. On the Phase A grid a typical query expands a few thousand cells and returns in a few milliseconds — fast enough to replan on every mission event rather than clinging to a stale plan.

2) Two robustness measures around the search: The textbook algorithm assumes the start and goal are free cells. In a running system, neither can be assumed, and both violations were observed within the first hour of testing.

a) Nearest-free snapping: The robot's own cell is frequently *inside* the inflation band — that is what standing next to a wall means once obstacles have been grown by the robot radius — and A* started from an occupied cell returns no

path at all. Likewise, a crate detected by the camera sits on the floor beside a basin, and its cell is often inflated. The planner therefore snaps both endpoints to the closest free cell by an expanding square-ring search:

```
for radius in range(1, max(rows, cols)):
    for dr in range(-radius, radius + 1):
        for dc in range(-radius, radius + 1):
            if max(abs(dr), abs(dc)) != radius:
                continue                    # ring,
    ↪ not disc
            if self._is_free(cand):
                return cand
```

The `continue` keeps the scan on the ring boundary, so cells are visited in increasing Chebyshev distance and the first free cell found is the nearest one. Snapping the goal is what makes "drive to the crate" mean "drive as close to the crate as the configuration space allows" rather than "fail".

b) Collinear smoothing: A raw A* result is one waypoint per cell: a 6 m path is sixty waypoints, fifty-eight of which lie on straight segments. The follower does not benefit from them and the mission log becomes unreadable. `_smooth` keeps a cell only when the step direction changes:

```
if (dx1, dy1) != (dx2, dy2):
    out.append(cells[i])
```

Sixty waypoints typically collapse to five or six, with the geometry of the path unchanged — this is a pure compression of the representation, not a shortcut. Genuine shortcutting (line-of-sight smoothing across the grid, as in Theta*) was considered and rejected: it produces paths that graze the inflation boundary, and the inflation margin is the only collision guarantee the system has.

3) Following the plan: pure pursuit: The follower is pure pursuit [**?**], [**?**]. Its state is a *cursor* on the polyline, (i, t), meaning "fraction t along segment i". Each tick:

1) The cursor is advanced to the orthogonal projection of the robot onto the polyline, and *never allowed to move backwards* (`self._cursor_t = max(self._cursor_t, t)`). Without that monotonicity, a robot pushed sideways past a corner will re-acquire an earlier segment and drive the path twice.
2) A target is taken `lookahead_distance` $= 0.4$ m further along the polyline, walking segment by segment so the distance is measured *along the path*, not as a chord.
3) A velocity command toward that target is issued, braked proportionally within `brake_distance` $= 0.6$ m of the final waypoint but never below `min_speed` $= 0.12$ m/s.
4) Success is tested against the *final waypoint* and the `position_tolerance` $= 0.15$ m, not against the cursor. Testing the cursor makes arrival depend on how the path happened to be discretised.

The lookahead distance is the single tuning knob, and it trades two failure modes against each other: too short and the robot oscillates about the path, over-correcting on every tick; too long and it cuts corners, which in a greenhouse means clipping a basin. The value 0.4 m was found by bisection against the 1.2 m aisle width of the Phase A arena, and it is worth noting that the Phase B greenhouse, with 0.80 m aisles, required it to be reduced to 0.32 m — the parameter is a property of the environment, not of the algorithm.

4) The holonomic command law — and the defect hidden in it: Phase A exploits the mecanum base directly. The velocity toward the lookahead target is computed in the world frame, rotated into the body frame, and issued *together with* a proportional yaw command that softly turns the base to face the direction of travel:

$$v_x^{\text{body}} = \cos\psi\, v_x^w + \sin\psi\, v_y^w, \tag{20}$$

$$v_y^{\text{body}} = -\sin\psi\, v_x^w + \cos\psi\, v_y^w, \tag{21}$$

$$\omega_z = \text{clip}(k_\psi\, \text{wrap}(\text{atan2}(\Delta y, \Delta x) - \psi),\ \pm 1.5)\,. \tag{22}$$

This is legal, elegant and, in the Phase A arena, effective: the base translates immediately along the path while its heading catches up, so no time is lost turning, and the lidar and arm end up pointing where the robot is going. It was validated in `nav_test` and used for the whole of Phase A without complaint.

It also contains a defect that Phase A was structurally incapable of revealing, and that cost several days in Phase B. Because translation and rotation are commanded simultaneously, the body points one way and travels another — it *crabs*. Two consequences follow:

- **The robot becomes wider.** A 0.58×0.38 m body crossing an aisle at $45°$ sweeps 0.68 m of it instead of 0.38 m. In a 1.2 m Phase A aisle that is harmless. In a 0.80 m Phase B aisle it is fatal.
- **The protective stop is aimed at the wrong obstacle.** Phase B's guard tests the corridor swept *along the commanded direction*; when that direction is oblique, the test rectangle points into the gutter beside the robot instead of down the lane ahead of it, and the brake fires for something the robot was never going to reach.

The measured cost in Phase B was **46%** of mission time spent braked inside lanes the robot could have driven straight down. The correction — turn to face the target, then translate along the body x axis scaled by $\cos(\text{heading error})$, reserving holonomic translation for the two manoeuvres that genuinely need it — is presented with its measurements in Sec. VIII. It is recorded *here*, in the Phase A chapter, because that is where the defect was written, and because it is the clearest example in this project of a design that is correct in one environment and wrong in another for reasons no amount of code review would surface. Only a narrower corridor and a time measurement exposed it.

5) Validation: `planner_test` builds a grid from the supervisor, plans between fixed start/goal pairs, and verifies that the returned polyline traverses only free cells — the property that inflation is supposed to guarantee. `nav_test` then drives those plans and reports final position error against the GPS. Both are single-purpose controllers with no mission logic,

which is what makes a failure in either of them attributable to one subsystem.

K. Manipulation: the Five-DOF Arm

[Section in preparation.]

L. Mission Supervision with Behaviour Trees

[Section in preparation.]

M. Phase A Results and Lessons Carried Forward

[Section in preparation.]

V. Phase B — ROS 2 / Gazebo Digital Twin: Overview

[Section in preparation.]

A. The Digital Twin: SDF World and URDF Robot

[Section in preparation.]

B. Software Architecture and the lib/ Boundary

[Section in preparation.]

C. The Protective Stop: a Single-Owner Safety Guard

[Section in preparation.]

D. Localisation: Odometry Calibration and a Homemade SLAM

[Section in preparation.]

E. Fruit Detection and Camera-to-Map Localisation

[Section in preparation.]

F. Mission: Coverage Survey then Harvest

[Section in preparation.]

G. Instrumentation: Measuring Pose, Map and Time

[Section in preparation.]

H. The Pre-Flight Regression Suite

[Section in preparation.]

VI. SWOT Analysis of Every Method

[Section in preparation.]

VII. Experimental Results

[Section in preparation.]

VIII. Discussion and Limitations

[Section in preparation.]

IX. Conclusion and Future Work

[Section in preparation.]

Appendix A
Reproducing Every Experiment

[Appendix in preparation.]

Appendix B
Complete Parameter Tables

[Appendix in preparation.]

Appendix C
Topic and Frame Contract

[Appendix in preparation.]

Appendix D
Failure Modes and Diagnosis

[Appendix in preparation.]

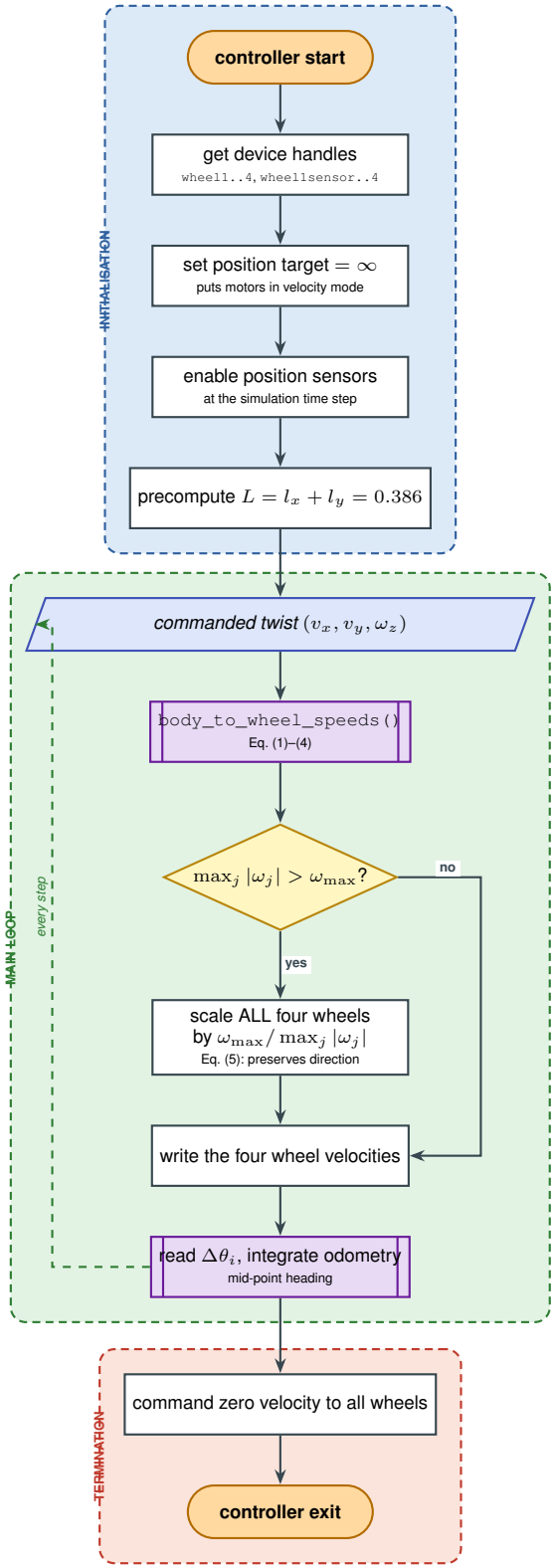
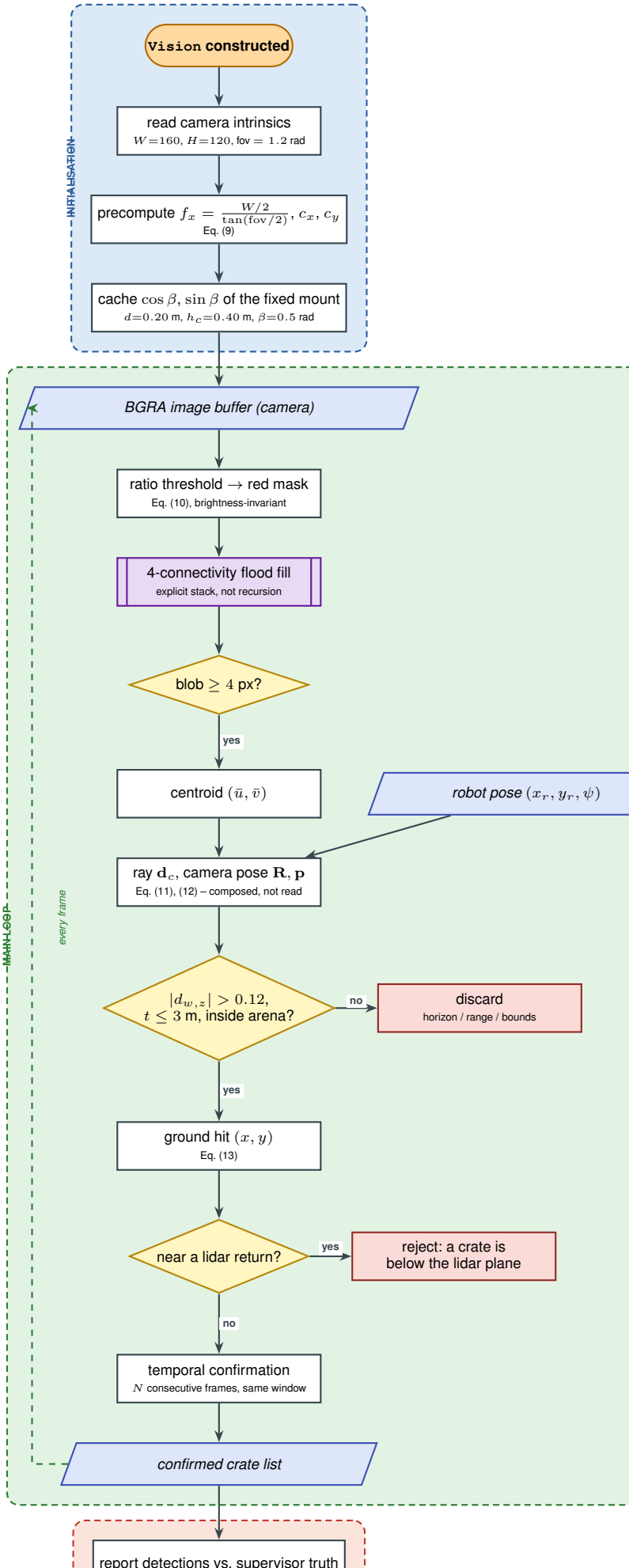
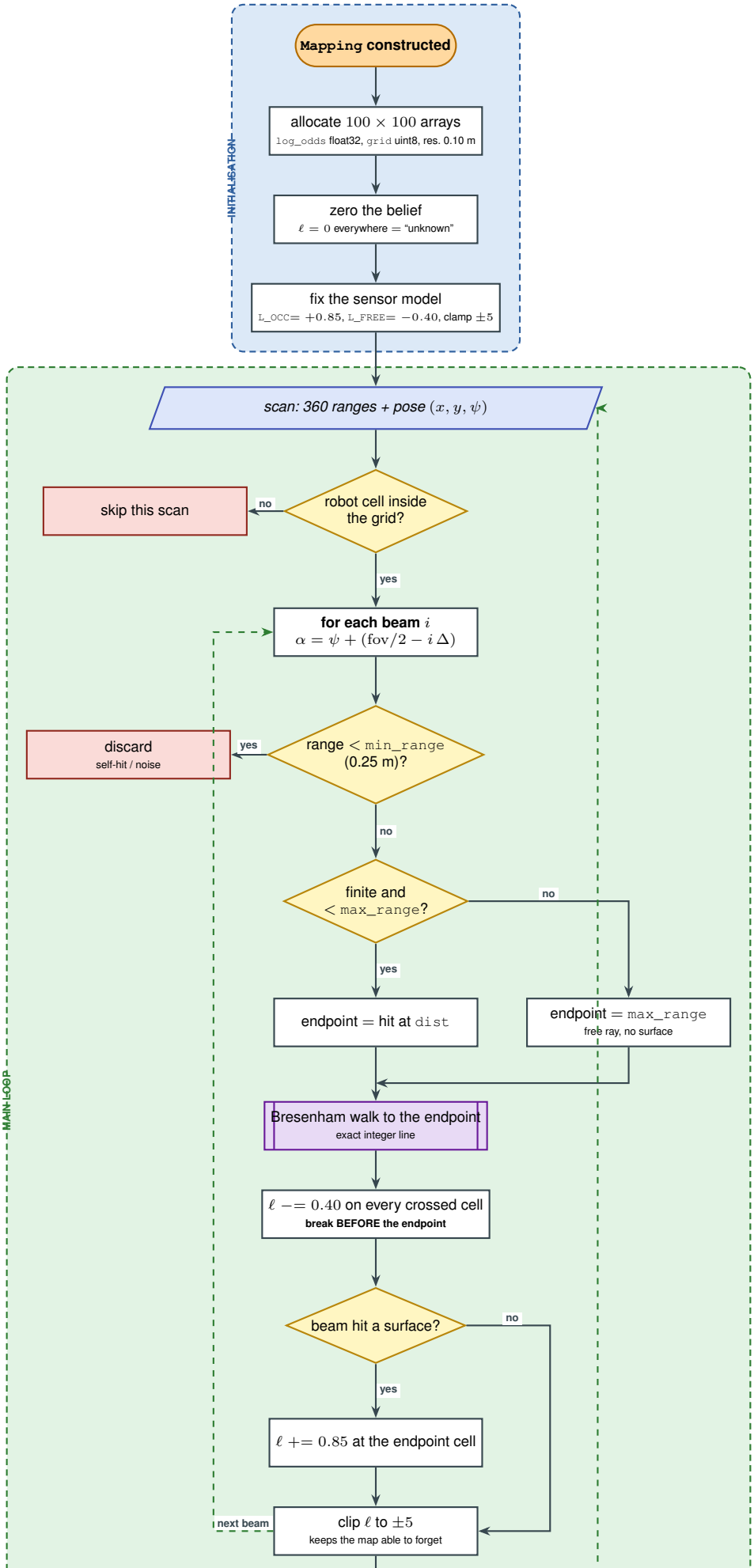
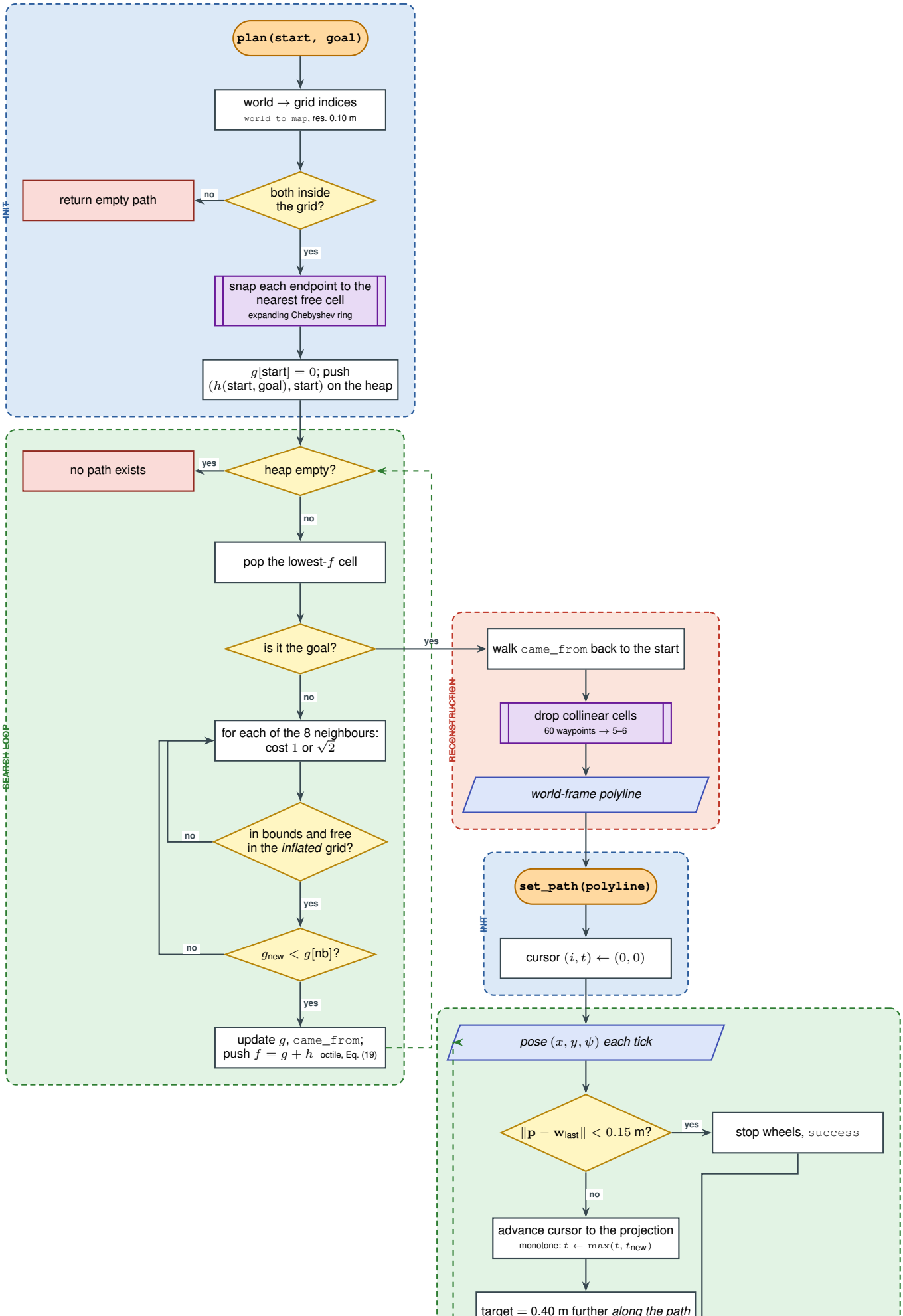


Figure 2: Mecanum base controller (Phase A). The initialisation band holds everything computable once, including the Webots idiom of setting a motor's position target to infinity to obtain velocity control. The saturation test is the only branch in the loop, and it scales all four wheels together so that a speed limit costs magnitude and never direction. The termination band exists because a controller that exits without zeroing the wheels leaves the base coasting into the next run.




INITIALISATION
Mapping constructed
allocate 100×100 arrays
log_odds float32, grid uint8, res. 0.10 m
zero the belief
$\ell = 0$ everywhere = "unknown"
fix the sensor model
L_OCC= $+0.85$, L_FREE= -0.40, clamp ± 5
MAIN LOOP
scan: 360 ranges + pose (x, y, ψ)
robot cell inside the grid?
no
skip this scan
yes
for each beam i
$\alpha = \psi + (\text{fov}/2 - i\,\Delta)$
range < min_range (0.25 m)?
yes
discard
self-hit / noise
no
finite and < max_range?
no
yes
endpoint = hit at dist
endpoint = max_range
free ray, no surface
Bresenham walk to the endpoint
exact integer line
ℓ −= 0.40 on every crossed cell
break BEFORE the endpoint
beam hit a surface?
no
yes
ℓ += 0.85 at the endpoint cell
clip ℓ to ± 5
keeps the map able to forget
next beam


plan(start, goal)
world → grid indices
world_to_map, res. 0.10 m
both inside the grid?
no
return empty path
yes
snap each endpoint to the nearest free cell
expanding Chebyshev ring
g[start] = 0; push (h(start, goal), start) on the heap
INIT
heap empty?
yes
no path exists
pop the lowest-f cell
is it the goal?
yes
no
for each of the 8 neighbours: cost 1 or √2
in bounds and free in the inflated grid?
no
yes
g_new < g[nb]?
no
yes
update g, came_from; push f = g + h octile, Eq. (19)
SEARCH LOOP
walk came_from back to the start
drop collinear cells
60 waypoints → 5–6
world-frame polyline
RECONSTRUCTION
set_path(polyline)
cursor (i, t) ← (0, 0)
INIT
pose (x, y, ψ) each tick
‖p − w_last‖ < 0.15 m?
yes
stop wheels, success
no
advance cursor to the projection
monotone: t ← max(t, t_new)
target = 0.40 m further along the path